

Spring Boot Security with OAuth2 & JWT

Modern security approaches for Spring Boot applications.

Discover how to protect APIs with industry-standard protocols.

Learn the difference between authentication and authorization in distributed systems.

 by Saratha Natarajan





Spring Security Fundamentals

76%

Fortune 500

Companies using Spring Security

100+

Features

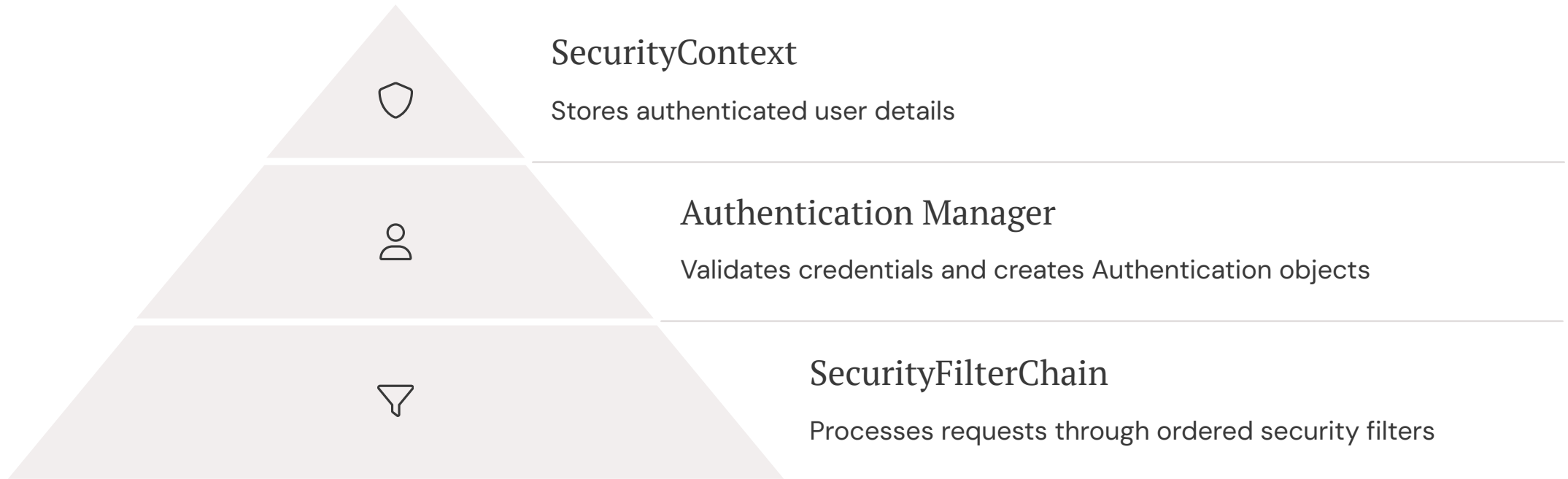
Built-in security capabilities

15+

Years

Of enterprise security protection

Spring Security Architecture



Authentication vs. Authorization

Authentication

Verifies who you are.

- Username/password validation
- Multi-factor authentication
- Biometric verification
- Token validation

Authorization

Determines what you can access.

- Role-based permissions
- Resource access control
- Function-level security
- Data filtering

Introduction to OAuth 2.0



OAuth2 Authorization Server



Client Registration

Register applications with client ID and secret



Authentication

Validate user identity through login forms



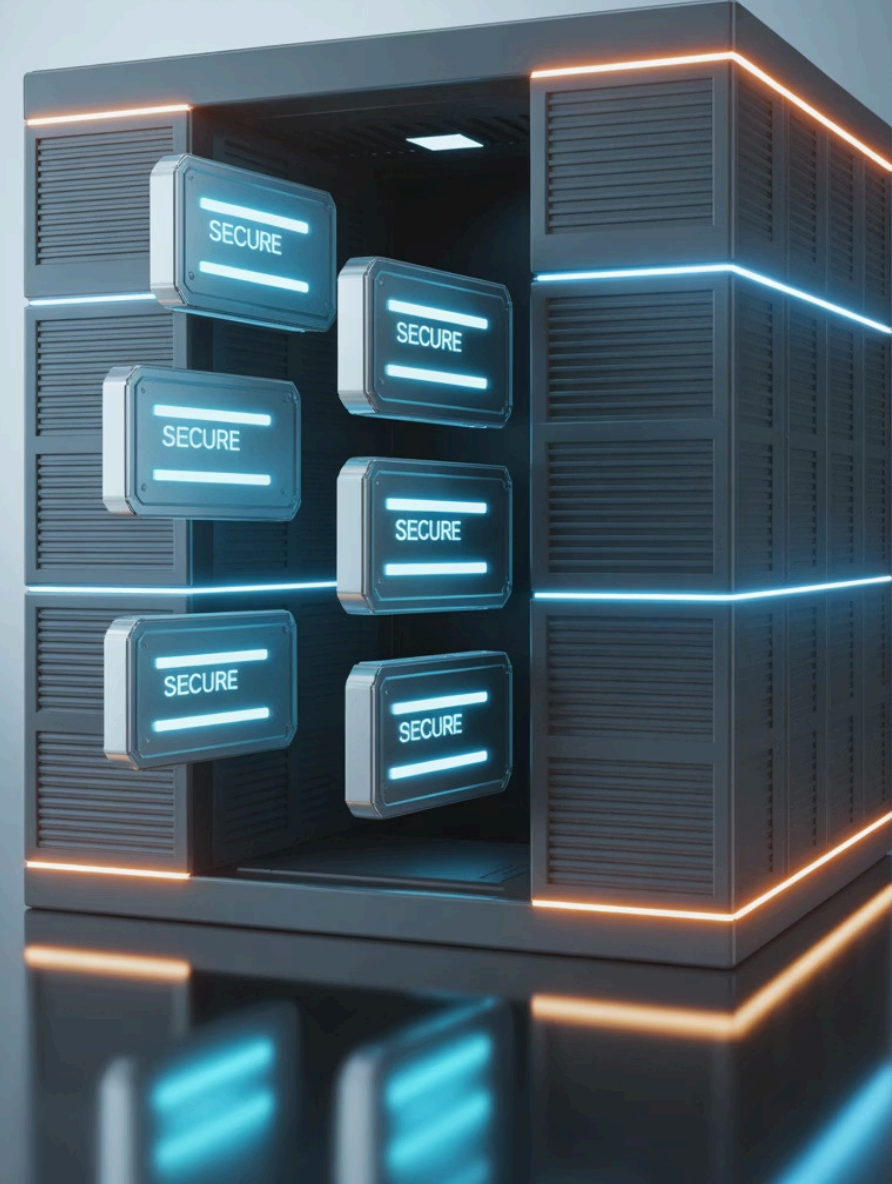
Authorization

Confirm resource access permissions



Token Issuance

Generate and return access/refresh tokens



A graphic featuring a glowing blue shield with a white center, positioned in front of a server rack. The shield has a grid of small dots. The server rack is dark blue with glowing blue lines and a small screen displaying three dots. The background is a gradient of orange and blue.

API Gateway

OAuth2 Resource Server

Token Validation

Verifies token signatures against trusted keys. Checks expiration dates and issuer claims.

Claims Extraction

Pulls user details and permissions from token. Converts claims to Security Context objects.

Resource Protection

Guards API endpoints based on extracted permissions. Denies access to unauthorized requests.

JWT (JSON Web Tokens)



Header

Contains token type and signing algorithm (e.g., RS256)



Payload

Holds claims about the user and token metadata



Signature

Verifies token hasn't been tampered with



Spring Boot with JWT Implementation

Add Dependencies

Include spring-security-oauth2-resource-server and spring-security-oauth2-jose in pom.xml

Configure JWT Decoder

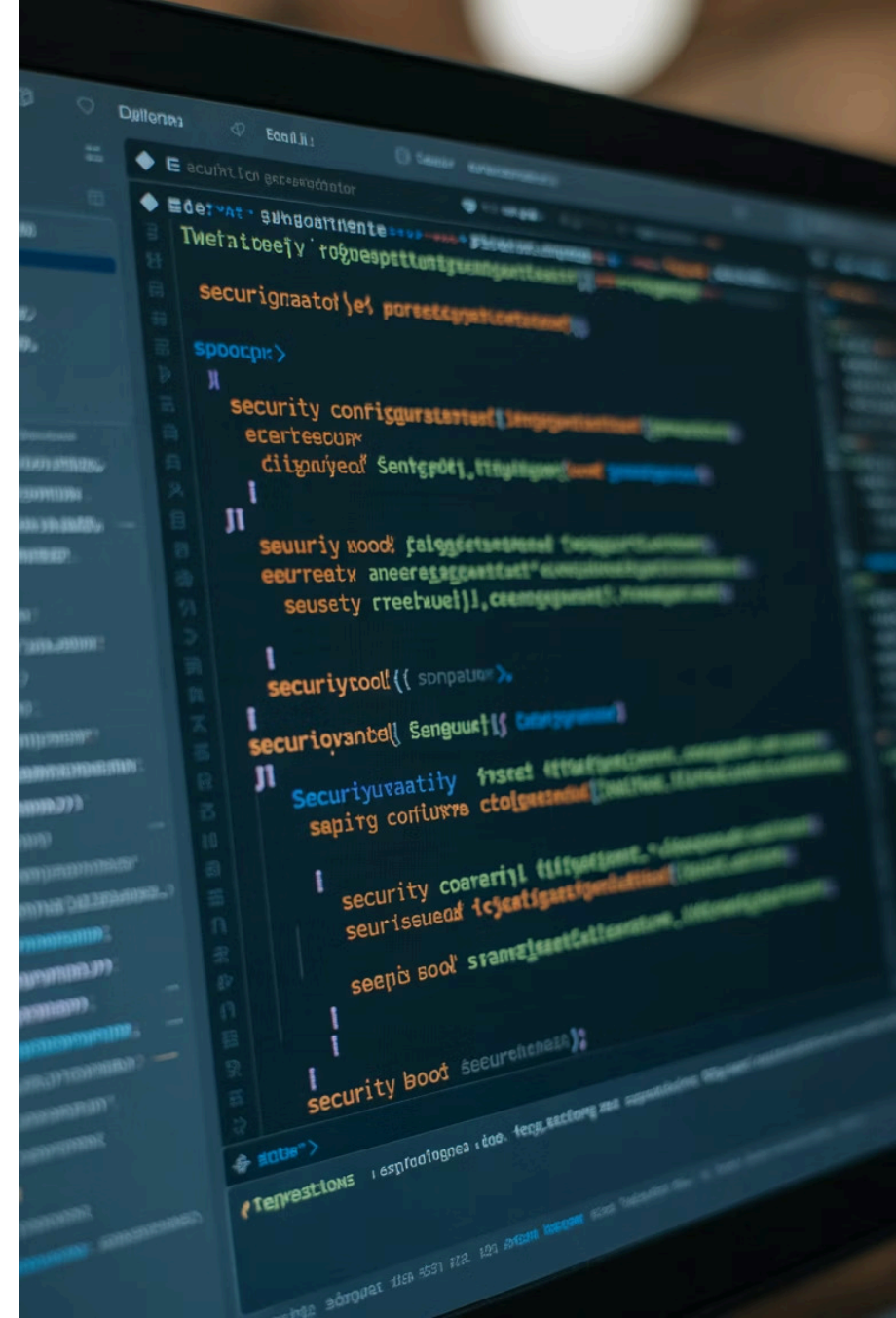
Set up NimbusJwtDecoder with RSA public key or JWKS URL

Create Security Config

Enable Resource Server and customize JWT conversion

Test Protection

Verify endpoints require valid JWT tokens



OAuth2 Client Implementation



Client Registration

Configure OAuth provider details



Authorization Request

Redirect users to login



Token Acquisition

Exchange authorization code for tokens



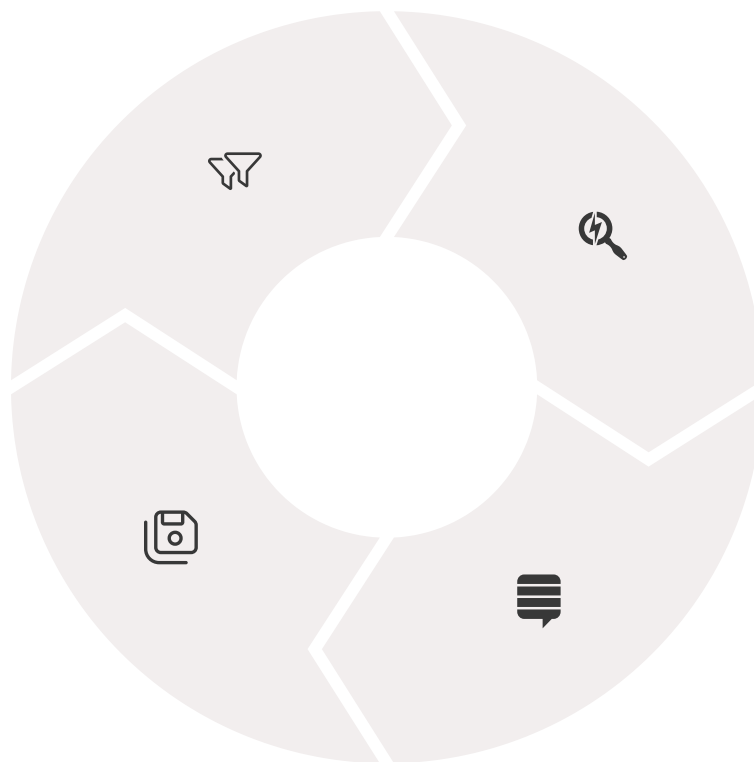
User Info Mapping

Convert provider data to user profile

JWT Authentication Filter

Extract Token
Pull JWT from Authorization header

Store Authentication
Save in SecurityContext for filters



Validate Token
Check signature and expiration

Convert Claims
Transform JWT to Authentication object

Securing REST APIs

Endpoint	Method	Security Expression
/api/users	GET	hasRole('ADMIN')
/api/profile	GET	hasRole('USER')
/api/orders	POST	hasAuthority('CREATE_ORDER')
/api/public	GET	permitAll()



OAuth2 + JWT Best Practices



Short Lifetimes

Keep access tokens valid for minutes, not days



Token Rotation

Use refresh tokens for obtaining new access tokens



Scope Limiting

Request only permissions your application needs



Secure Storage

Never store tokens in localStorage or cookies without protection

Security Best Practices Checklist



Cocceochctuien
Prees tie die chichele



Thnd Siowich
Prarterice e bahennjerten



Tnnontracloaf
Baferawioe lvenold



Baclned Schgon
Pacotad cle vilosi



Toonee Bipusistect
Precis der yiengicci



Pupemdündtouner
Ebcstives uaiijisd



Oocecotadived
Preeerveiaded eaiconclc



Raihenr caddeden
Pfeerveiereodereded



Meementoulheht
Pceeoögite cspicerteige



Scorneatc.ta
Pronratict's Uenofoee



Tohanis Uto'culy
Festent wias tieceliug



Marianttind tuatien
Preenectiee tinckosece

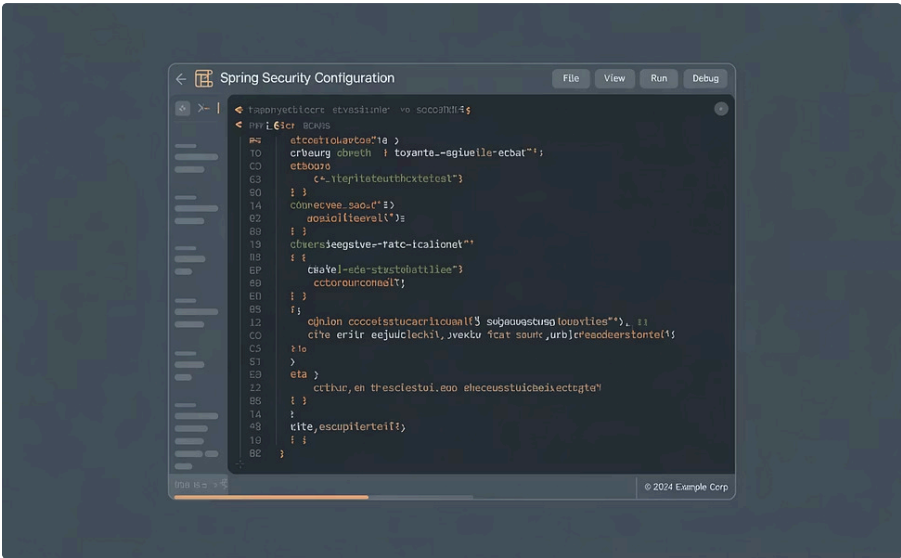


Tyctkecessine
Pmie wñhoientestioe



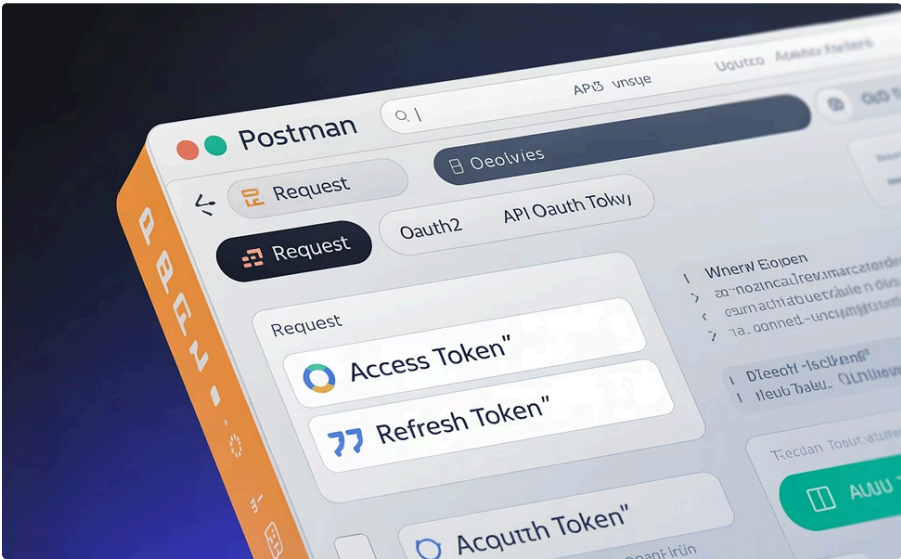
Mocuelind íocaluine
Fetredt velriied

Demo: Complete Implementation



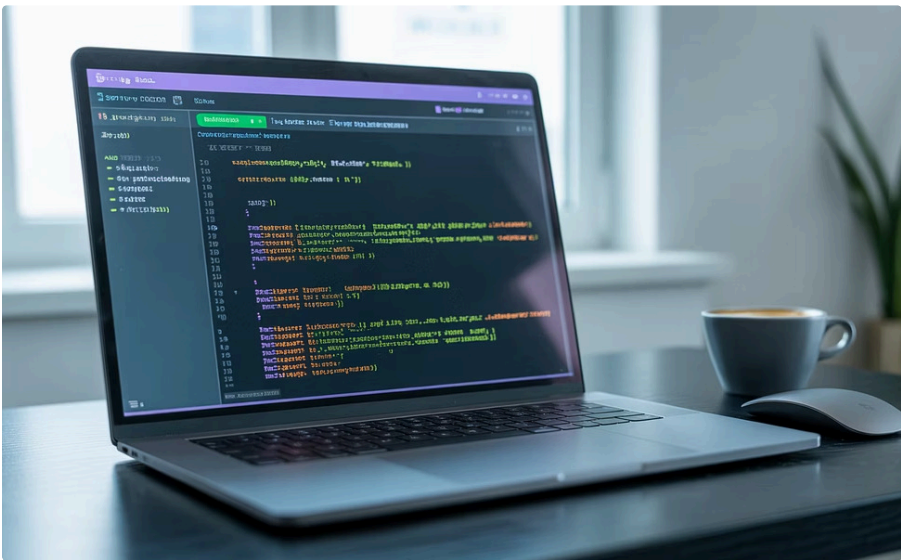
Source Code

Complete GitHub repository with working example application



API Testing

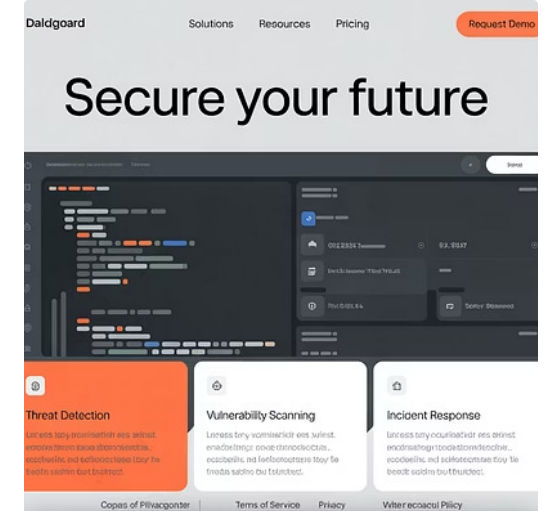
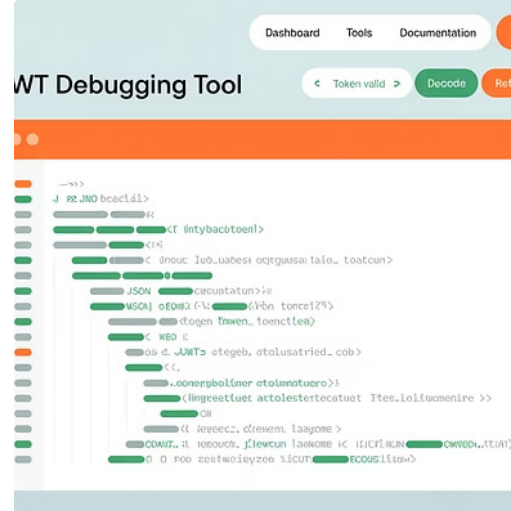
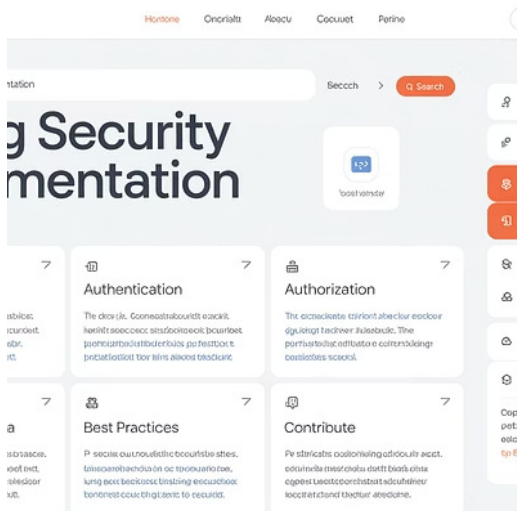
Test your implementation with Postman collections



Runtime Verification

Monitor your application's security behavior

Resources & Next Steps



Expand your knowledge with official Spring Security documentation and guides.

Explore advanced topics like multi-tenancy, PKCE, and device flows.